Fill The Cup

Developer manual | SatisVizion | Version 1.0.0

Unity 6000.0.62f1 - Universal Render Pipeline

Contents

1. Overview and included content
2. Requirements and installation
3. First play, controls and progression
4. Project structure and exact source files
5. Fill The Cup reskin: interface colours
6. Fill The Cup reskin: menu, HUD and tutorial text
7. Boosters, charges, unlocks and first use
8. Artwork, fonts, audio and branding
9. Visual editor: open, inspect and save
10. Automatic generation and difficulty tuning
11. Data schema and safe manual edits
12. Saving, testing profiles and developer controls
13. LiveOpsKit setup, simulated ads and optional providers
14. Android build and release preparation
15. WebGL build, hosting and browser checks
16. Troubleshooting and practical verification
17. Licences, customization handoff and support

Overview and included content

Draw lines with the available ink to guide falling water into the cup. Reach the dashed fill target while managing the level layout and drawing budget.

Fixed-step water simulation, drawn surfaces, limited ink, authored cup layouts, helpers, visual editing and seeded level generation.

This is an editable Unity template for developers. The supplied game is a working starting point; use the included source, assets and authoring tools to produce your own title. The template contains 25 authored levels. The game keeps its required private runtime Kit and the publisher-owned HyperCasual LiveOps Kit together in one product folder.

The shipping service configuration uses simulated Dummy ads and Disabled IAP. Advertising, billing, analytics and other optional provider SDKs are not bundled or automatically installed. A simulation is useful for testing callbacks and user-interface flow; it does not establish live advertising revenue, purchases, cloud storage or a production backend. The Android APK is a debug-signed showcase, not your release signing key or a marketplace submission.

Read the installation steps before pressing Play. Read the reskin recipes before replacing colours or text, and use the level editor before changing JSON by hand. Both portrait and landscape entry scenes are supplied. The offline HTML and PDF manuals contain the same practical guidance, so a purchaser can begin before visiting the hosted documentation.

Requirements and installation

1. Extract the main download completely into a normal writable folder. Keep the native Unity package, source project and offline documentation available while setting up. Back up an existing project before importing a new template.

2. Use Unity `6000.0.62f1`. In Unity Hub, install Android Build Support with its matching SDK, NDK and OpenJDK for Android builds, and WebGL Build Support for browser builds. Do not substitute a different NDK simply because another application installed one.

3. For the supplied standalone source project, choose Add project in Unity Hub and select the `Unity Project` directory containing `Assets`, `Packages` and `ProjectSettings`. Do not select the enclosing storefront-material folder as a Unity project.

4. For package import, create a Universal Render Pipeline project, open `Assets > Import Package > Custom Package` and import the complete native package. Retain `Assets/SatisVizionFillCup`, including its game, private Kit, HyperCasualLiveOpsKit, TextMesh Pro, fonts, settings and metadata. Import into a clean project first when diagnosing an integration problem.

5. Check `Window > Package Manager` for Universal RP `17.0.4`, Unity UI `2.0.0` and Input System `1.14.2`. The standalone manifest also lists the standard Unity modules used by the template, including `com.unity.modules.screencapture` and `com.unity.modules.androidjni`. These are Unity dependencies; optional advertising and billing packages are separate.

6. Open `Edit > Project Settings > Player > Other Settings > Active Input Handling`, select `Both` and restart the Editor if Unity requests it. This is a manual prerequisite. The buyer template does not secretly inspect or change that setting during import, domain reload or Play mode.

7. Allow package resolution and script compilation to finish. Resolve red Console errors before opening several tools at once. A missing Input System namespace usually means its package did not resolve; an input exception often means the Both setting or restart was missed.

8. In Project Settings > Graphics, assign the supplied `FillCup_URP.asset` from `Assets/SatisVizionFillCup/Games/FillCup/Settings`. Check the pipeline assignment for the active Quality level as well. Retain the matching renderer and generated global/volume settings. A missing renderer or wrong pipeline can produce pink materials or a blank world.

9. Open `Assets/SatisVizionFillCup/Games/FillCup/Scenes/FillCup_Mobile.unity` for portrait. Press Play, focus the Game view and follow the first introduction. Use the landscape `Assets/SatisVizionFillCup/Games/FillCup/Scenes/FillCup_WebGL.unity` scene for the alternate layout. Stop Play before changing persistent source or assets.

10. Save the project after setup. Record the engine/package versions that worked and keep a clean copy. If you later upgrade Unity or URP, treat that as a separate migration: back up, update deliberately and repeat the gameplay, authoring and build checks.

First play, controls and progression

Draw a surface using press/drag/release on the board. When a line is committed, the fixed-step simulation releases water from the configured taps. Guide it into the cup to reach the dashed fill mark; inspect the cup progress and any level-specific mechanics. Ink is a length budget, so an unnecessarily long line may leave too little for another surface. Obstacles and no-draw areas restrict valid strokes; hot, sponge, jelly, conveyor, paddle and sugar objects have their own simulation behavior. Hint previews the stored solution; Eraser removes an eligible existing stroke; Ink Pot adds half of the original allowance; Slow-Mo slows eligible active flow for six seconds at a 0.33 factor. Hint needs an available solution before flow, Eraser needs a stroke, and Slow-Mo needs flowing water with particles still to spawn. Test an effect in the correct state rather than treating every disabled button as a broken integration. Portrait and landscape entry scenes share a portrait-oriented game column rather than promising an entirely different wide-screen puzzle.

Draw lines with the available ink to guide falling water into the cup. Reach the dashed fill target while managing the level layout and drawing budget.

Start with the opening content rather than a late challenge selected from developer controls. Read the goal and any first-time mechanic hint, perform the indicated gesture, and watch the goal counters change. The tutorial is part of the real game flow. A control may remain unavailable while a popup, introduction, animation or guided step is active; confirm or complete that step before interpreting a tap as broken input.

The current goal determines success. Check the displayed remaining shots, time, moves, tray capacity, opponents or picture progress as appropriate for this game. A successful result advances the normal progression and invokes the configured reward/ad hooks. Failure follows the game-specific retry or recovery flow. Developer jump, unlock and charge controls exist for testing; they are not evidence that a challenge was completed through normal play.

Test a new visual theme in the opening scene and a later challenge. Small counters, disabled helpers, result dialogs, pause/settings, long hint text and the alternate orientation often reveal layout problems that a title screen does not. Confirm that a UI element is still readable against the background, has a useful touch target and does not cover the play area. Keep essential colour distinctions visible after a palette change.

Progress, first-use hints, unlocked content and helper charges are stored locally by the game. Stop Play before editing authored assets. A runtime change to a generated UI object usually disappears when the scene is rebuilt. Use the documented source assignments, level editor and save controls to make a permanent change.

Project structure and exact source files

```
Assets/SatisVizionFillCup/
  Games/FillCup/
    Scenes/
    Scripts/
    Editor/
    Resources/fillcup/
    Settings/
    Kit/
    Documentation/
  HyperCasualLiveOpsKit/
  TextMesh Pro/
  Documentation/
```

The game root for every file below is `Assets/SatisVizionFillCup/Games/FillCup`. Keep the private Kit with this game; it supplies its runtime foundation and editor/build support. The service kit is at `Assets/SatisVizionFillCup/HyperCasualLiveOpsKit`. Text resources remain at `Assets/SatisVizionFillCup/TextMesh Pro`. Do not merge another exported game Kit into this product or remove required assembly definitions to silence an error.

File relative to game root	Purpose
`Scripts/FcArt.cs`	Artwork lookup and code-defined palette.
`Scripts/FcBot.cs`	Automated search or simulation used by the game/editor.
`Scripts/FcData.cs`	Serializable definitions and shared constants.
`Scripts/FcGen.cs`	Seeded content generation.
`Scripts/FcGfx.cs`	Game-specific implementation; inspect its public types and callers before changing its data contract.
`Scripts/FcLevels.cs`	Level storage, definitions or authoring helpers.
`Scripts/FcSfx.cs`	Sound names and playback helpers.
`Scripts/FcSim.cs`	Game-specific implementation; inspect its public types and callers before changing its data contract.
`Scripts/FillCupGame.cs`	Game lifecycle, goals, input, progression and helper handlers.
`Scripts/FillCupGame.Hud.cs`	HUD creation, counters, helper presentation and displayed captions.
`Scripts/FillCupGame.View.cs`	Rendering, materials, geometry and scene presentation.

Unity references many imported objects through the GUID stored in their .meta file. Move or rename an asset in Unity where possible, and keep metadata in version control. Replacing a texture while preserving its GUID can retain serialized links, but renaming a Resources base name still requires updating the matching code lookup. A GUID and a Resources string solve different reference problems.

Fill The Cup reskin: interface colours

The interface is built by C# when the scene runs. It is not one permanent scene prefab whose final labels can all be edited in the Hierarchy. Start with these exact colour declarations and their actual consumers. A game palette may include both UI colours and world colours; changing one does not automatically replace pixels already baked into a sprite.

Exact declaration	Current value	UI consumer locations
`Scripts/FcArt.cs:10`	`Paper = #FBF7EE`	FillCupGame.Hud.cs:307
`Scripts/FcArt.cs:10`	`Grid = #D9E4EF`	FillCupGame.Hud.cs:158
`Scripts/FcArt.cs:10`	`GridBold = #C5D5E6`	Artwork/world palette; no direct HUD assignment found. Change interface constants with HUD consumers first.
`Scripts/FcArt.cs:10`	`Ink = #2B2D42`	FillCupGame.Hud.cs:58, FillCupGame.Hud.cs:70, FillCupGame.Hud.cs:82, FillCupGame.Hud.cs:120
`Scripts/FcArt.cs:11`	`Sage = #6FB7A8`	FillCupGame.View.cs:156
`Scripts/FcArt.cs:11`	`SageDark = #4E9787`	Artwork/world palette; no direct HUD assignment found. Change interface constants with HUD consumers first.
`Scripts/FcArt.cs:11`	`Coral = #FF7F6E`	FillCupGame.cs:290, FillCupGame.Hud.cs:93, FillCupGame.Hud.cs:225, FillCupGame.Hud.cs:269
`Scripts/FcArt.cs:11`	`Sun = #FFC94A`	FillCupGame.Hud.cs:266, FillCupGame.Hud.cs:287, FillCupGame.View.cs:543
`Scripts/FcArt.cs:11`	`Water = #4FB6F0`	FillCupGame.Hud.cs:161, FillCupGame.Hud.cs:269, FillCupGame.View.cs:290
`Scripts/FcArt.cs:12`	`WaterRim = #2E8FD0`	Artwork/world palette; no direct HUD assignment found. Change interface constants with HUD consumers first.
`Scripts/FcArt.cs:12`	`Lime = #A8D64A`	Artwork/world palette; no direct HUD assignment found. Change interface constants with HUD consumers first.
`Scripts/FcArt.cs:12`	`LimeDark = #7FB22E`	FillCupGame.cs:124, FillCupGame.cs:265, FillCupGame.Hud.cs:283, FillCupGame.View.cs:98
`Scripts/FcArt.cs:12`	`Hot = #F0583C`	FillCupGame.View.cs:150
`Scripts/FcArt.cs:12`	`HotGlow = #FFB25B`	Artwork/world palette; no direct HUD assignment found. Change interface constants with HUD consumers first.
`Scripts/FcArt.cs:13`	`Sponge = #FFD86B`	FillCupGame.View.cs:151

Exact declaration	Current value	UI consumer locations
`Scripts/FcArt.cs:13`	`SpongeWet = #B9C9A0`	Artwork/world palette; no direct HUD assignment found. Change interface constants with HUD consumers first.
`Scripts/FcArt.cs:13`	`Jelly = #FF8FC8`	FillCupGame.View.cs:152, FillCupGame.View.cs:529
`Scripts/FcArt.cs:13`	`Belt = #5C6378`	FillCupGame.View.cs:153
`Scripts/FcArt.cs:13`	`Steel = #9AA3B5`	FillCupGame.View.cs:212, FillCupGame.View.cs:224, FillCupGame.View.cs:227
`Scripts/FcArt.cs:14`	`Porcelain = #FFFFFF`	Artwork/world palette; no direct HUD assignment found. Change interface constants with HUD consumers first.
`Scripts/FcArt.cs:14`	`Sub = #7C8197`	FillCupGame.Hud.cs:60, FillCupGame.Hud.cs:72, FillCupGame.Hud.cs:285, FillCupGame.Hud.cs:348
`Scripts/FcArt.cs:14`	`Locked = #C9CDD8`	FillCupGame.Hud.cs:266, FillCupGame.Hud.cs:279
`Scripts/FcArt.cs:14`	`Card = #FFFFFF`	Artwork/world palette; no direct HUD assignment found. Change interface constants with HUD consumers first.
`Scripts/FcArt.cs:14`	`Sky = #4C9EEB`	FillCupGame.cs:124, FillCupGame.Hud.cs:394, FillCupGame.Hud.cs:422
`Scripts/FcArt.cs:15`	`Plum = #8E7CC3`	FillCupGame.Hud.cs:393, FillCupGame.Hud.cs:395
`Scripts/FcArt.cs:15`	`NoDraw = #FF7F6E`	FillCupGame.View.cs:154
`Scripts/FcArt.cs:15`	`Sugar = #FFFFFF`	FillCupGame.View.cs:197
`Scripts/FcArt.cs:15`	`Cookie = #E9A75F`	FillCupGame.View.cs:155

Worked example: edit `Assets/SatisVizionFillCup/Games/FillCup/Scripts/FcArt.cs`, find `Ink` and replace only its hex value. The following line is taken from this game:

```
// Before
public static readonly Color Paper = HzUI.Hex("#FBF7EE"), Grid = HzUI.Hex("#D9E4EF"), GridBold =
HzUI.Hex("#C5D5E6"), Ink = HzUI.Hex("#2B2D42"),

// After
public static readonly Color Paper = HzUI.Hex("#FBF7EE"), Grid = HzUI.Hex("#D9E4EF"), GridBold =
HzUI.Hex("#C5D5E6"), Ink = HzUI.Hex("#385C8A"),
```

1. Stop Play and commit or back up the original file. Search for the exact symbol, not every occurrence of a similar colour in unrelated code. Keep the type, declaration name and surrounding syntax intact.

2. Replace `#2B2D42` with `#385C8A`. A six-digit value supplies RGB; an eight-digit value also carries alpha. If the original value includes alpha, preserve its intended transparency unless you deliberately redesign the overlay.

3. Save and let Unity recompile. Reopen the portrait scene and inspect the consumers named above, then repeat in landscape. Check both a normal HUD and an open settings/result popup.

4. Adjust related foreground/background colours together until counters, buttons, disabled states and focus/hint markers remain readable. Keep game-rule colour IDs, matching indices and serialized values unchanged during a cosmetic reskin.

5. If the colour does not change, verify that you edited this product copy, that the relevant UI path uses the symbol, and that a later refresh is not applying another tint. If the asset is a baked sprite, replace the sprite artwork rather than repeatedly changing an unrelated palette constant.

Fill The Cup reskin: menu, HUD and tutorial text

Use the exact game assignments below. The main `Title` property is in `Assets/SatisVizionFillCup/Games/FillCup/Scripts/FillCupGame.cs`, but separately constructed title art and HUD labels may use additional literal strings. Shared-kit localization does not automatically translate hard-coded game captions. Changing a localization table alone will leave these strings unchanged.

File and line	Method / context	Current assignment
`Scripts/FillCupGame.cs:23`	class declarations	`public override string Title => "Fill The Cup";`
`Scripts/FillCupGame.cs:30`	class declarations	`public static readonly string[] BoostName = { "Hint", "Eraser", "Ink Pot", "Slow-Mo" };`
`Scripts/FillCupGame.cs:124`	ShowIntro	`FcPopup("New!", L.hint, Sky, MechanicArt(), L.name, -1, ("Got it", LimeDark, (Action)Begin));`
`Scripts/FillCupGame.cs:265`	LevelWon	`FcPopup(stars == 3 ? "Perfect Pour!" : "Cup Filled!", msg, LimeDark, null, null, stars,`
`Scripts/FillCupGame.cs:290`	LevelLost	`FcPopup(best >= 0.5f ? "So Close!" : "Oh No!", msg, Coral, null, null, -1, buttons.ToArray());`
`Scripts/FillCupGame.cs:417`	OnBooster	`ShowTip(BoostName[b] + " unlocks at level " + BoostUnlock[b], 2.1f); HzAudio.Play(Sfx.Cancel, 0.4f);`
`Scripts/FillCupGame.cs:477`	OfferCharge	`buttons.Add(("Watch ad", Sky, () => HzPlatform.Rewarded(GameId, "booster_" + BoostName[b].ToLowerInvariant(), ok => { if (ok) GiveCharge(b); })));`
`Scripts/FillCupGame.cs:479`	OfferCharge	`FcPopup("+1 " + BoostName[b], "Out of " + BoostName[b] + ".\nYou have " + Coins + " coins.", HzUI.Hex(BoostColor[b]), Get(BoostArt[b]), null, -1, buttons.ToArray());`
`Scripts/FillCupGame.Hud.cs:93`	BuildSideDecor	`var title = HzUI.Label("Title", _layer, "Fill The Cup", 76, Coral, TextAnchor.MiddleCenter);`

File and line	Method / context	Current assignment
Scripts/FillCupGame.Hud.cs:184	SlowBadge	_slowTxt = HzUI.Label("T", b.transform, "SLOW-MO", 38, Color.white); _slowTxt.rectTransform.Stretch();
Scripts/FillCupGame.Hud.cs:239	RefreshHud	_levelTxt.text = Spaced("LEVEL " + _playLevel.ToString(CultureInfo.InvariantCulture));
Scripts/FillCupGame.Hud.cs:240	RefreshHud	_nameTxt.text = L.name + (L.breather ? " - breather" : "");
Scripts/FillCupGame.Hud.cs:284	RefreshBoosters	v.Caption.text = un ? BoostName[b] : "Level " + BoostUnlock[b];
Scripts/FillCupGame.Hud.cs:386	UnlockPopup	FcPopup("New: " + BoostName[b] + "!", BoostDesc[b] + "\nYou get " + BoostStart[b] + " free!", HzUI.Hex(BoostColor[b]), Get(BoostArt[b]), null, -1,
Scripts/FillCupGame.Hud.cs:393	ShowSettingsPopup	var p = FcPopup("Settings", "", Plum, null, null, -1,
Scripts/FillCupGame.Hud.cs:422	HelpPage	FcPopup($"How to Play {i + 1}/{HelpPages.Length}", HelpPages[i], Sky, null, null, -1,

Worked visible-text edit in `Assets/SatisVizionFillCup/Games/FillCup/Scripts/FillCupGame.Hud.cs`, method `RefreshHud`:

```
// Before
_levelTxt.text = Spaced("LEVEL " + _playLevel.ToString(CultureInfo.InvariantCulture));

// After
_levelTxt.text = Spaced("Stage " + _playLevel.ToString(CultureInfo.InvariantCulture));
```

1. Stop Play, open the named source file and find the actual assignment. Change the displayed string occurrence shown in the example, not the preceding object/control name in a Label or Button call. An object called `Level` can keep that name even when its visible caption becomes Stage.

2. Preserve interpolation expressions, concatenated counters, formatting calls and escaped `\n` line breaks. For example, keep a numeric `{...}` expression intact when changing the words around it. Do not translate resource paths, save keys, helper IDs, question IDs or assembly names as though they were display text.

3. Search for the same visible wording in refresh methods as well as initial construction. A later `.text =` assignment can replace an initial label every frame or after loading another level. Update each intended portrait/landscape variant and test after a result screen.

4. For popup headings, settings, help and tutorial wording, use the matching file/method rows above. Level or opponent `name`, `hint`, introductions and answer text can also come from the editor/data rather than a C# literal. Preserve the data schema while changing those captions.

5. Re-enter Play with the first introduction and relevant first-use state available. Test long replacements, line breaks, narrow portrait cards, button height, missing glyphs and the result dialog. A translated sentence that fits the landscape HUD can overflow a phone-width popup.

For a multi-language edition, route these game-specific assignments through your chosen translation layer and supply translations for the actual keys used by this game. Retain a fallback language. Verify plural forms, digits, fonts and input rules separately. Do not globally replace every quoted token: many quoted values select art, sounds, scene objects, rule variants or persistent storage.

Additional exact popup, help and settings captions: these rows cover this game's custom card/button factories and its actual private-Kit settings implementation. Edit the displayed literal while retaining callbacks, interpolation and object/resource identifiers. Reopen the relevant dialog after recompiling.

Exact source location	Method	Visible-caption assignment
`Scripts/FillCupGame.cs:149`	Begin	`else ShowTip(L.breather ? " Breather: an easy one!" : "Fill the cup up to the dashed line", 2.5f);`
`Scripts/FillCupGame.cs:393`	StartWater	`if (_playLevel == 1 && ! TutorialDone) ShowTip("Great! Now watch the water flow.", 2.2f);`
`Scripts/FillCupGame.cs:401`	NoInk	`ShowTip(Unlocked(BInk) ? "Out of ink! Try the Ink Pot" : "Out of ink!", 2.0f);`
`Scripts/FillCupGame.cs:427`	OnBooster	`if (_hintOn) { ShowTip("Trace the dotted line!", 2.0f); return; }`
`Scripts/FillCupGame.cs:431`	OnBooster	`ShowTip("Trace the dotted line!", 2.2f);`
`Scripts/FillCupGame.cs:434`	OnBooster	`if (S.Strokes.Count == 0) { ShowTip("Draw a line first - the Eraser takes it back", 2.3f); HzAudio.Play(Sfx.Cancel, 0.4f); return; }`
`Scripts/FillCupGame.cs:438`	OnBooster	`ShowTip("Line erased - ink refunded!", 1.8f);`
`Scripts/FillCupGame.Hud.cs:455`	StartTutorial	`ShowTip(IsPortrait ? "Draw a line so the water reaches the cup!" : " Draw a line so the water\nreaches the cup!\nFollow the hand.");`
`Kit/Runtime/HzGame.cs:243`	ShowSettings	`list.Add((HzAudio.Muted ? "Sound: OFF" : "Sound: ON", HzUI.Hex("#4C9EEB"), null));`
`Kit/Runtime/HzGame.cs:244`	ShowSettings	`list.Add((HzAudio.MusicMuted ? " Music: OFF" : "Music: ON", HzUI.Hex("#4C9EEB"), null));`
`Kit/Runtime/HzGame.cs:245`	ShowSettings	`if (onRestart != null) list.Add(("Restart", HzUI.Hex("#F5A623"), onRestart));`
`Kit/Runtime/HzGame.cs:246`	ShowSettings	`if (onHome != null) list.Add(("Home", HzUI.Hex("#7B61FF"), onHome));`

Exact source location	Method	Visible-caption assignment
`Kit/Runtime/HzGame.cs:247`	ShowSettings	`list.Add(("Close", HzUI.Hex("#3DBE6E"), null));`
`Kit/Runtime/HzGame.cs:248`	ShowSettings	`var p = Popup("Settings", "", HzUI.Hex("#4C9EEB"), list.ToArray());`

Boosters, charges, unlocks and first use

Each helper belongs to this game, even though the common kit supplies supporting UI and services. The order of its name, artwork, unlock and grant arrays must stay aligned. A locked helper displays its required progression step. Newly available helpers introduce themselves through the game flow; a guided use can be handled differently from a normal charged use, so inspect the spend and guide-completion handlers before changing the economy.

Helper	Unlock step	Effect / source description	Initial free charges
Hint	3	Shows a dotted line that works. Trace it to fill the cup!	2
Eraser	5	Takes back your last line and gives its ink back.	2
Ink Pot	8	Adds 50% more ink for this try.	2
Slow-Mo	12	Slows the water down for 6 seconds so you can draw in time.	1

Exact source location	Definition
`Scripts/FillCupGame.cs:28`	`public static readonly int[] BoostUnlock = { 3, 5, 8, 12 };`
`Scripts/FillCupGame.cs:29`	`public static readonly int[] BoostStart = { 2, 2, 2, 1 };`
`Scripts/FillCupGame.cs:30`	`public static readonly string[] BoostName = { "Hint", "Eraser", "Ink Pot", "Slow-Mo" };`
`Scripts/FillCupGame.cs:53`	`bool Unlocked(int b) => HzDev.BoostersUnlocked || _playLevel >= BoostUnlock[b];`
`Scripts/FillCupGame.cs:477`	`buttons.Add(("Watch ad", Sky, () => HzPlatform.Rewarded(GameId, "booster_" + BoostName[b].ToLowerInvariant(), ok => { if (ok) GiveCharge(b); })));`
`Scripts/FillCupGame.Hud.cs:284`	`v.Caption.text = un ? BoostName[b] : "Level " + BoostUnlock[b];`
`Scripts/FillCupGame.Hud.cs:375`	`static readonly string[] BoostDesc =`

To rename a helper, edit the full-name and description definitions in the exact file listed here. Some games keep names in the main Game class and descriptions in the HUD, UI, Popups or data file. Do not assume both are in one file. Preserve the array length/order and IDs. For the board games, display names and descriptions are returned by the helper data methods rather than a common `BoostName` array.

1. Reach the unlock step through normal play or use the developer jump/unlock controls only for testing. Confirm the introduction and initial charge count.

2. Activate the helper in a state where it has a useful target. A button can legitimately reject use while a popup, tutorial, animation or incompatible mode is active.

3. For a targeted helper, finish its second input, such as selecting a board cell, tray piece or picture region. Arming a helper is not the same as demonstrating its effect. For undo, first perform a reversible move.

4. Read the charge badge before and after the action. Test both an available charge and an exhausted-charge offer. In Dummy mode, a rewarded-ad simulation can exercise the completion callback; it is not a paid transaction or a live ad impression.

5. Change durations, amounts or grants in the actual game handler/constants and retest progression. Avoid granting charges on every refresh; grants should follow their intended first-use, reward or purchase event. Keep rewarded placement IDs stable when changing only visible copy.

The developer panel can unlock helpers and grant charges to speed testing. Use a fresh local profile to validate the real first-use journey as well. A successful developer activation does not prove that the original unlock progression or exhausted-charge callback still works.

Artwork, fonts, audio and branding

Game artwork and sound live under `Assets/SatisVizionFillCup/Games/FillCup/Resources/fillcup`. The exact resource folders are: `Resources/fillcup/Art`, `Resources/fillcup/Levels`, `Resources/fillcup/Music`, `Resources/fillcup/Sfx`. UI tint constants do not replace artwork already baked into an image. Generated meshes and materials can use different constants from HUD cards, so inspect the view/material code as well as the palette table.

1. Replace an existing sprite or clip in Unity while preserving the matching .meta/GUID. Keep its Resources base name unless you also change every load call. Import sprite replacements with the appropriate texture type, transparency, filtering and pixels-per-unit settings. Check an icon at its actual small HUD size rather than only at full resolution.

2. For 3D assets or procedural materials, inspect the game view/world class and any material setup menu. Back up customized materials before rerunning a setup/generation command. Check lighting, shadows, readability and performance on the target build; a polished Editor screenshot alone is not a device test.

3. The private Kit resolves its fonts in `Kit/Runtime/HzUI.cs`. Update the actual font references or lookup paths when changing a font. Retain redistribution notices for supplied fonts and choose replacements with suitable usage rights. Verify every translated glyph and text weight.

4. Audio lookups and cues are defined in the game sound helper and `Kit/Runtime/HzAudio.cs`. Preserve clip keys and GUIDs when replacing recordings. Use suitable source WAV assets, leave headroom and test music/sound toggles independently. Audio conversion cannot recover fidelity already lost in an earlier compressed recording.

5. Change the company, product name, application identifier, icons and release settings in Project Settings > Player for your own title. Changing the in-game Title property does not update all platform branding. Check the title screen, settings/help, launcher icon, install name and web-page title separately.

Marketing covers and gameplay captures have different purposes. An illustrated cover can communicate the theme, but it is not a screenshot of a new game feature. Keep storefront claims tied to the shipped product and replace promotional images after a major reskin. Keep legitimate source-asset licences and notices with redistributed files.

Visual editor: open, inspect and save

Fill The Cup preview controls and worked edit: Open Tools > Fill The Cup > Level Editor. Select a level, choose an object or drawing tool, and edit tap, cup and shape fields alongside the visual preview. The editor supports drawing/testing solution strokes and running the actual game simulation; a decorative curve is not automatically a valid solution. Duplicate a simple level, move one obstacle slightly, then Find solution or edit its stored solution and validate the water result, fill fraction and ink usage. Preserve target cup geometry and a reachable fill mark. Auto Generate has Count 1-50, First level number, Seed, Start/End difficulty 0-1, Curve shape, Sawtooth 0-0.3 and Breather every 0-10 (zero disables breathers). First level number influences generation numbering/difficulty/seed; it is not the replacement index. With Replace from selection off, generated levels append at the end of the file. With it on, replacement begins at the currently selected row and continues through existing rows, inserting beyond the end if necessary. Try Count 3, First level number 26, seed 4242, replace off in a 25-level backup. The generator validates with the bot and can use a proven fallback layout when a candidate fails; review the report instead of claiming every requested candidate survived. Validate All and test actual drawing/flow before saving.

Open `Tools > Fill The Cup > Level Editor`. Its implementation is `Assets/SatisVizionFillCup/Games/FillCup/Editor/FillCupLevelEditor.cs`. This is a Unity Editor authoring tool; it is not a playable in-game level editor. Stop Play before editing persistent content. The editor reads the game definitions from `Assets/SatisVizionFillCup/Games/FillCup/Resources/fillcup/Levels/levels.json`.

Controls present in this game editor
Undo
Reload
Validate
Validate All
Find solution
Play this level
Add
Dup
Del
Up
Down
Level
Name

Controls present in this game editor
First-time hint
Ink budget
Difficulty
Breather
Solution ink
Ink = solution x 2.5
Clear solution
Validation (bot draws through the game simulation)
Auto Generate
First level number
Seed
Start difficulty
End difficulty
Curve shape
Sawtooth
Replace from selection
Selected: tap
X
Y
Pours
Interval (s)
Speed
Delete tap
Selected: cup
X (centre)
Y (bottom)
Mouth width
Height
Fill mark

1. Select an existing non-tutorial entry and inspect its preview and fields. For a first customization, duplicate that entry using its actual Duplicate/Dup control rather than delete or overwrite the opening tutorial.

2. Rename the duplicate in its Name field. Make one small change at a time. Start with a visible caption or a modest existing budget/goal value that is genuinely present in this editor; do not add a guessed JSON field.

3. Use the editor preview and its documented gestures to change geometry or content. Follow the concrete preview controls and worked edit described above.

4. Run Validate for the selected entry, then read its result. Fix invalid data or a failed solver/bot check before relying on the preview. Use Validate All after changing shared rules, generation constants or a group of entries.

5. Save explicitly. An unsaved preview is not the file the game will load after restart. Respect the dirty indicator/save prompt and use the editor Undo/Redo controls where supplied. Reload replaces unsaved in-memory changes from disk.

6. Enter Play and select the edited entry through normal progression or the developer jump control. Check the opening hint, actual goal, win/fail behavior and both orientations. Stop Play, reload the editor and confirm the saved data persists.

Duplicate, move and delete commands change authored order. Because progression and helper introductions refer to steps, reordering entries can alter pacing and unlock timing even when each board remains valid. Keep the opening onboarding sequence coherent. Back up the JSON before bulk edits and retain a known-good copy for comparison.

Automatic generation and difficulty tuning

Exact Fill The Cup insertion/replacement behavior: Replace from selection OFF appends at the end. ON starts at the currently selected row, overwrites existing entries sequentially and inserts new entries after the old range ends. First level number affects generation numbering/seed/curve, not the replacement position. Save remains separate.

These are the actual field bindings and limits in `Assets/SatisVizionFillCup/Games/FillCup/Editor/FillCupLevelEditor.cs`. Sliders list their minimum and maximum in the source call; integer fields are additionally clamped where shown. Defaults below are the source defaults, not a promise that an already-open editor has no saved UI state.

Visible control	Field	Control/limits from source	Location
Count	`_genCount`	`IntSlider(_genCount, 1, 50)`	`Editor/FillCupLevelEditor.cs:463`
First level number	`_genStart`	`IntField(_genStart)`	`Editor/FillCupLevelEditor.cs:464`
Seed	`_genSeed`	`IntField(_genSeed)`	`Editor/FillCupLevelEditor.cs:465`
Start difficulty	`_genFrom`	`Slider(_genFrom, 0, 1)`	`Editor/FillCupLevelEditor.cs:466`
End difficulty	`_genTo`	`Slider(_genTo, 0, 1)`	`Editor/FillCupLevelEditor.cs:467`
Curve shape	`_genCurve`	`CurveField(_genCurve)`	`Editor/FillCupLevelEditor.cs:468`

Visible control	Field	Control/limits from source	Location
Sawtooth	`_genWave`	`Slider(_genWave, 0, 0.3f)`	`Editor/FillCupLevelEditor.cs:469`
Breather every	`_genBreather`	`IntSlider(_genBreather, 0, 10)`	`Editor/FillCupLevelEditor.cs:470`
Replace from selection	`_genReplace`	`Toggle(_genReplace)`	`Editor/FillCupLevelEditor.cs:471`

```
line 47: int _genCount = 10, _genStart = 26, _genSeed = 4242, _genBreather = 5;
line 48: float _genFrom = 0.25f, _genTo = 0.85f, _genWave = 0.1f;
```

Use the game editor generation controls rather than manually manufacturing an unchecked board. The generator and bot/simulation are game-specific; a seed selects reproducible input, but changing algorithm constants or engine behavior can change the result for the same seed. Record the seed and parameters when reporting a difficult generated challenge.

Generation-related controls detected in this editor
Difficulty
Breather
Auto Generate
Seed
Start difficulty
End difficulty
Replace from selection

1. Back up the authored JSON and choose the intended generation section/mode.

2. Set a small Count first, then the starting entry/index and Seed shown in the actual editor. Use its design-difficulty or start/end/ramp controls where available. Keep early onboarding entries intact.

3. Read the replacement control before generating. If the actual control says Replace range, selected existing entries may be overwritten; an insert/append mode instead adds content. Do not treat these words as interchangeable. Generate into a safe range or a copied file first.

4. Inspect the new previews and run validation. Read failures and measured difficulty, not only the requested difficulty label. Simulation/search limits and different goal types can produce different measurements.

5. Play representative easy, middle and hard generated entries. Include a breather and any new mechanic/helper introduction. Check that the generator has not produced an impossible target, inaccessible item, illegal position or budget that contradicts the intended goal.

6. Save only after review. Reopen the file, repeat a representative generation with the same seed when testing reproducibility, and compare the expected structural fields. Keep a separate record of intentional generator-code changes.

Validation is a development aid with the scope implemented by this particular game. A bot pass does not replace human usability testing, and a numerical difficulty score is not a universal measure across games. When a rule or helper changes, rerun both authored-content checks and representative procedural continuation. Do

not bypass a validation failure by merely changing its stored score.

Data schema and safe manual edits

The main data file is `Assets/SatisVizionFillCup/Games/FillCup/Resources/fillcup/Levels/levels.json`. Its top-level authored collections are `levels` (25). The following keys and sample values come from this actual game file; they are not a generic schema borrowed from another template.

Key in first authored entry	JSON value type	Current sample
`name`	string	`"First Pour"`
`hint`	string	`"Draw a line so the water reaches the cup!"`
`breather`	boolean	`false`
`difficulty`	number	`0.05`
`ink`	number	`13`
`seed`	number	`1`
`taps`	array	`[{"x": -1.8, "y": 4.2, "dir": 0, "count": 72, "interval": 0.05, "speed": 2.2}]`
`cups`	array	`[{"x": 1.6, "y": -4.9, "w": 1.5, "h": 1.6, "mark": 0.7}]`
`shapes`	array	`[{"kind": "box", "x": 1.6, "y": -5.03, "w": 2, "h": 0.24, "rot": 0, "p": 0}]`
`solution`	array	`[{"pts": [{"x": -2.6, "y": 1}, {"x": 0.8, "y": -0.4}]}]`
`botStars`	number	`3`
`botInk`	number	`3.6769567`
`botFill`	number	`1`
`score`	number	`-1`

Use the corresponding serializable data classes in the source reference to determine which fields are editable inputs and which are calculated or compatibility metadata. In particular, capitalized totals such as Max/Total can be derived convenience values, not tuning controls. Prefer the editor for board coordinates, object relationships, shape references, positions, answer banks and other structured values.

1. Stop Play and close or reload the authoring window before a manual file edit. Make a backup and keep the original UTF-8 JSON.

2. Preserve exact field spelling/case, array structure, numeric types and quoted strings. JSON does not support comments or trailing commas. Escape literal quote characters and line breaks correctly.

3. Change one existing visible name/hint or a verified editable field. Do not rename a resource, gameplay identifier, helper ID, rule variant or saved progression key as part of a cosmetic text edit.

4. Reopen/reload the editor and read parsing or validation errors. Run selected-entry and collection-wide checks before saving or building.

5. Play the edited content in both entry scenes. Verify its actual completion/retry behavior and any associated tutorial or helper use. A file that parses successfully can still describe an invalid or impractical challenge.

Saving, testing profiles and developer controls

The game persists local progression and settings through its game/Kit save helpers. Search the exact game ID `fillcup` and `Assets/SatisVizionFillCup/Games/FillCup/Kit/Runtime/HzCore.cs` when tracing a save. Visible labels are not storage keys. Renaming keys can make existing progress disappear or cause first-use flows and charge grants to repeat; treat a save-key change as a migration, not a reskin.

The supplied developer panel supports rapid testing, such as jumping to content, unlocking helpers, granting charges and controlling simulated service behavior. Its button and panel are designed to draw above game popups. Use these controls for a targeted check, then test again with normal progression and a fresh profile. Never include a recording made with forced completion as evidence that a puzzle was solved normally.

Reset only a test profile you intend to clear. A broad PlayerPrefs reset can remove settings and progression for the current application. On devices, uninstalling/reinstalling or changing the application identifier may also affect which local profile is used. Record the profile state when reporting an issue, especially when the symptom concerns a first-time hint, a previously unlocked helper or a reward granted only once.

For a public release, review `Assets/SatisVizionFillCup/Games/FillCup/Kit/Runtime/HzDev.cs` and its `HZ_RELEASE` build symbol. The source documents how the developer panel and overrides are disabled for release. Confirm the result in a real release build; hiding a visible DEV button alone is not the same as disabling development behavior. Keep development-only controls out of the player-facing store release unless you deliberately support them.

LiveOpsKit setup, simulated ads and optional providers

The included service kit is at `Assets/SatisVizionFillCup/HyperCasualLiveOpsKit`. Its settings asset is `Assets/SatisVizionFillCup/HyperCasualLiveOpsKit/Resources/HLK/HLKSettings.asset`. Keep its resources, assembly definitions and runtime adapters together. The game routes service events through its private Kit, including `Assets/SatisVizionFillCup/Games/FillCup/Kit/Runtime/HzCore.cs`. Do not remove that code because the project does not contain a vendor advertising SDK.

Setting	Shipping configuration	Meaning
Ads Mode	`Dummy (0)`	Simulated banner, interstitial and rewarded flows; no live ad-network request.
IAP Mode	`Disabled (3)`	Purchasing is off. No live store billing is established by the template.
Provider SDKs	Not included; manual setup	Install and configure an optional official provider separately if you choose to use it.

Setting	Shipping configuration	Meaning
Native-only capabilities	Platform/configuration dependent	Editor/WebGL simulation does not prove native sharing, notifications, reviews or device APIs.

1. Open the actual kit setup/settings window under Tools > HyperCasual LiveOps Kit and inspect the configured modes. Keep Dummy ads and Disabled IAP while checking the supplied no-SDK edition.

2. Play a normal game flow that requests a banner/interstitial/rewarded placement. The simulation should be clearly labelled. Exercise completion, skip/close and unavailable outcomes where supported, and check that gameplay resumes and rewards are applied only through the intended callback.

3. If you choose a real provider, follow the official provider documentation linked by the kit. Install its supported SDK separately and configure your own application IDs, placements, dashboard, platform settings and test environment. Opening Package Manager is not an automatic provider installation.

4. Keep production credentials and release signing material outside redistributed templates and public repositories. Review provider terms, consent, privacy disclosures and service costs for your own release. Do not put another publisher's production IDs into the asset.

5. Test the actual target platform after configuration. A rewarded simulation, synthetic revenue event or successful Editor callback does not prove a real advertisement, payment, notification delivery or native sharing result.

The kit also contains optional progression, reward, shop and other system demonstrations. These are included capabilities that require your own game design and configuration; they are not a promise that every example service is already enabled or connected to a production backend. Read the kit customer documentation at `Assets/SatisVizionFillCup/HyperCasualLiveOpsKit/Documentation/Documentation.html` for its broader setup guidance.

Android build and release preparation

Build the supplied no-SDK demo: after checking your settings, use `Tools/Fill The Cup/Build Android` from Unity's menu. The actual implementation is `Assets/SatisVizionFillCup/Games/FillCup/Kit/Editor/HzTools.cs`, method `BuildAndroid()` through `Build(string target)`. It writes `Builds/Android/fillcup.apk` relative to the project folder. It selects the correct game scenes, keeps unused shared-kit showcase resources out of the player and restores those resources after building; the buyer project retains them.

For Dummy/Test ads this game-specific command deliberately enters the kit's local simulated-build scope, so the non-development demonstration can build without a live advertising provider. The normal production readiness guard remains active outside that explicit scope; File > Build Profiles > Build can therefore report missing production-provider configuration while Dummy mode is selected. Do not disable the guard or install a vendor SDK just to reproduce the supplied simulated demo. With Ads Mode Production, configure and test your own supported provider and release settings; the game-specific command keeps production checks active.

1. Use the matching Unity Android support toolchain and open File > Build Profiles. Select Android and allow Unity to complete its target switch and reimport. Keep the required Unity modules resolved before building.

2. Choose the portrait game scene `Assets/SatisVizionFillCup/Games/FillCup/Scenes/FillCup_Mobile.unity` as the startup scene. Do not accidentally use a kit showcase or a temporary test scene. Inspect the scene list explicitly.

3. In Player Settings, set your company/product name, unique application identifier, icons, version and version code. Configure orientation and target architecture for the release you intend to distribute. The supplied showcase is not a substitute for those release decisions.

4. For final store delivery, keep Development Build and script debugging off. Configure your own signing key securely and back it up. A debug-signed demonstration APK is for trying the game; do not assume it is your production-signed store artifact.

5. Decide whether the destination requires an APK or Android App Bundle and build the appropriate output. Review any optional provider-specific Android configuration only after installing that provider deliberately. The no-SDK template does not silently acquire a dependency resolver or vendor SDK.

6. Install the resulting build on a supported Android device and check touch controls, portrait layout, safe areas, audio/vibration settings, pause/resume, saving, result/retry flow and representative helper uses. Test configured services in their intended test/production environment.

7. Repeat release checks after changing package versions, platform settings, signing, native plugins, consent flow or provider configuration. Keep the exact artifact hash and configuration record so you can identify which build was tested.

The public Android demo link is a purchaser trial, not your private signing material. If you redistribute a demonstration APK, verify that a person without your account can download it and that it corresponds to the current product version. A Google Drive owner-view link alone does not prove public download permission.

WebGL build, hosting and browser checks

Build the supplied no-SDK demo: after checking your settings, use `Tools/Fill The Cup/Build WebGL` from Unity's menu. The actual implementation is `Assets/SatisVizionFillCup/Games/FillCup/Kit/Editor/HzTools.cs`, method `BuildWebGL()` through `Build(string target)`. It writes `Builds/WebGL/fillcup/` relative to the project folder. It selects the correct game scenes, keeps unused shared-kit showcase resources out of the player and restores those resources after building; the buyer project retains them.

For Dummy/Test ads this game-specific command deliberately enters the kit's local simulated-build scope, so the non-development demonstration can build without a live advertising provider. The normal production readiness guard remains active outside that explicit scope; File > Build Profiles > Build can therefore report missing production-provider configuration while Dummy mode is selected. Do not disable the guard or install a vendor SDK just to reproduce the supplied simulated demo. With Ads Mode Production, configure and test your own supported provider and release settings; the game-specific command keeps production checks active.

1. Switch the Build Profile to WebGL with the matching Unity module installed. Decide which supplied game layout is the entry experience and set the startup scene explicitly. Include any additional scene your own navigation actually loads.

2. Use a supported WebGL template and review resolution, aspect ratio, canvas scaling and browser input. The game has distinct authored portrait and landscape scenes; changing a canvas width alone does not turn one into the other.

3. Build into a clean output directory. Keep the generated loader, data, framework and WebAssembly files together with their expected filenames and paths. Do not upload only index.html or rename compressed build files without updating the loader configuration.

4. Serve the build over HTTP/HTTPS. Opening index.html directly with a file URL is not a reliable WebGL test. Configure the host to serve the compression and MIME types required by your chosen Unity build settings, or use the appropriate decompression fallback.

5. Open the hosted demo in a fresh browser session and confirm that it finishes loading and accepts input. Test a first introduction, representative gameplay, a helper, a result/retry path and settings. Reload to check expected local save behavior.

6. Test the intended desktop and mobile browsers/layouts. Audio can require an initial user gesture; browser storage, focus, keyboard and native API behavior can differ from the Editor and Android. Do not claim unsupported native features are established by a WebGL simulation.

7. Check the actual public URL after deployment. A successful local build or Git push does not prove that the hosted page serves the current files. Keep a versioned artifact/checksum record and verify the final hosted bytes when refreshing a demo.

When embedding the demo, give the canvas usable space and provide clear navigation to the correct layout. Avoid clipping important controls beneath an iframe or browser overlay. Verify the link a purchaser will actually click, including its redirects and any query parameters. Keep documentation and Android trial links separate from temporary deployment or authenticated management URLs.

Troubleshooting and practical verification

Symptom	Check and next action
Compilation fails after import	Wait for package resolution, confirm the exact Unity/URP/uGUI/Input System versions, import the full product root and read the first Console error. Do not delete required Kit or assembly definitions to hide a dependency problem.
Input exception or unresponsive classic controls	Set Active Input Handling to Both manually and restart if prompted. Focus the Game view, close a blocking popup and test the correct gesture in the opening tutorial.
Pink, blank or incorrectly lit scene	Check the supplied URP asset and renderer in Graphics and the active Quality level. Restore missing Settings assets/GUIDs and confirm the intended entry scene.
Text reverts after a reskin	Find the refresh assignment as well as the constructor. Update the intended visible literal in the actual game file/method table, then test after loading another challenge.
Helper appears unavailable	Check its unlock step, charge count, current goal, first-use guidance and whether another popup/animation/helper is active. For undo, perform a reversible action first.
Changed JSON is ignored or rejected	Stop Play, confirm the exact Resources data path, reload the editor, fix JSON/schema errors and validate. Check whether an unsaved editor buffer overwrote your manual edit.
Expected ad SDK or native service is absent	This edition deliberately omits optional provider SDKs. Dummy ads and Disabled IAP are expected. Follow manual provider setup and test on a supported platform before claiming production functionality.
WebGL stops loading	Check the complete build upload, loader paths, compression/MIME configuration, browser Console and correct public URL. Test through HTTP/HTTPS, not a file URL.
Progress or tutorial state is unexpected	Identify the current application ID/test profile and saved flags. Reset only the intended test profile, then reproduce from the opening flow.

Symptom	Check and next action
Audio clips distort or go silent	Check source/import headroom, volume toggles, browser user-gesture requirements and the actual cue/resource key. Keep licences and metadata when replacing recordings.

A useful verification pass includes a clean import, both game scenes, a normal first-use profile, a later challenge, all four helpers, a failed/retry path, settings/audio/vibration, save/reload, the level editor, selected/all validation and representative generated content. Check the actual Android and browser builds separately. Record what you tested rather than treating an earlier or different-platform result as proof for the current artifact.

Licences, customization handoff and support

Keep the supplied legal and attribution notices for code, fonts, icons, sounds and other assets. The owner-supplied HyperCasual LiveOps Kit is included with this template; follow the applicable item and component licences when redistributing a customized game or sharing source. Optional provider SDKs have their own terms and are not bundled in this no-SDK edition. Do not assume that converting a file format removes an attribution or usage obligation.

After a reskin, keep a concise record of the source palette/text files you changed, the content seed/range you generated and the engine/package versions you tested. Rebuild purchaser trials and update promotional captures when the visible game changes. Retain the exact release archive and checksum that correspond to the listed version. Avoid distributing temporary capture helpers, editor automation, internal planning notes, credentials or private signing material as part of the buyer asset.

For product assistance, use the marketplace item support channel or contact `satisvizion@gmail.com`. Include the game title/version, Unity and package versions, target platform, the first relevant Console error, reproduction steps and whether the issue occurs in the unmodified template. For an editor/generated-content issue, include the affected entry or seed and the actual validation result. Remove secrets, signing keys and production identifiers before sharing a project or log.

This manual explains the supplied template and configuration. It does not guarantee a marketplace decision, a particular revenue result, compatibility with an untested engine upgrade or functionality of a provider that has not been separately installed and configured. Use the concrete file references and repeatable steps above to make and verify your own changes.